DBMS – SQL QUERY IMPLEMENTATION FOR IKEA E-COMMERCE DATABASE

Project Title: IKEA E-Commerce Database Management System
Skill Task: DBMS

Objectives

1. Perform SELECT, WHERE, ORDER BY, and DISTINCT queries.
2. Search products based on price, category, and availability.
3. Retrieve customer and product information.
4. Apply different SQL filtering conditions.
5. Generate basic business reports.

Sample data used

Product_ID	Product_Name	Price (₹)	Category_ID	Stock_Quantity
101	KALLAX Shelf Unit	4,999	7	25
102	MALM Bed Frame	12,999	2	18
103	LACK Coffee Table	2,499	1	20
104	POANG Armchair	6,999	1	32
105	MICKE Desk	5,999	5	15
106	INGOLF Dining Chair	3,999	4	0
107	FADO Table Lamp	1,499	8	40
108	MULIG Clothes Rack	1,999	7	12
109	APPLARO Outdoor Table	8,999	6	27
110	FEJKA Artificial Plant	799	10	0

1. SQL QUERIES

1.1 SELECT Query

The SELECT statement is used to retrieve data from one or more tables. The columns to be displayed are written after the SELECT keyword.

Query 1.1a: Display All Products

```
SELECT * FROM Product;
```

Only the required columns are listed, so the result shows the product ID, name, price and category ID of every product.

Query 1.1b: Display Selected Product Details

```
SELECT Product_ID, Product_Name, Price, Category_ID
FROM Product;
```

```
SQL> SELECT Product_ID, Product_Name, Price, Category_ID
  2  FROM Product;

PRODUCT_ID PRODUCT_NAME                PRICE CATEGORY_ID
---------- -------------------- ---------- -----------
       101 KALLAX Shelf Unit            4999           7
       102 MALM Bed Frame              12999           2
       103 LACK Coffee Table            2499           1
       104 POANG Armchair               6999           1
       105 MICKE Desk                   5999           5
       106 INGOLF Dining Chair          3999           4
       107 FADO Table Lamp              1499           8
       108 MULIG Clothes Rack           1999           7
       109 APPLARO Outdoor Table        8999           6
       110 FEJKA Artificial Plant        799          10

10 rows selected.
SQL>
```

1.2 WHERE Clause

The WHERE clause filters records so that only the rows satisfying a condition are returned.

This query returns the low-priced products whose Price is below 2000.

Query 1.2a: Products with Price Less Than ₹2,000

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Price < 2000;
```

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price < 2000;

PRODUCT_ID PRODUCT_NAME                PRICE
---------- -------------------- ----------
       107 FADO Table Lamp              1499
       108 MULIG Clothes Rack           1999
       110 FEJKA Artificial Plant        799

3 rows selected.
SQL>
```

This query returns the higher-priced products whose Price is above 5000.

Query 1.2b: Products with Price Greater Than ₹5,000

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Price > 5000;
```

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price > 5000;

PRODUCT_ID PRODUCT_NAME               PRICE
---------- -------------------- ----------
       102 MALM Bed Frame            12999
       104 POANG Armchair             6999
       105 MICKE Desk                 5999
       109 APPLARO Outdoor Table      8999

4 rows selected.
SQL>
```

)

1.3 ORDER BY

The ORDER BY clause sorts the result set. ASC sorts from lowest to highest (the default) and DESC sorts from highest to lowest.

The cheapest product appears first and the most expensive product appears last.

Query 1.3a: Products from Lowest to Highest Price

```
SELECT Product_ID, Product_Name, Price
FROM Product
ORDER BY Price ASC;
```

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  ORDER BY Price ASC;

PRODUCT_ID PRODUCT_NAME               PRICE
---------- -------------------- ----------
       110 FEJKA Artificial Plant       799
       107 FADO Table Lamp             1499
       108 MULIG Clothes Rack          1999
       103 LACK Coffee Table           2499
       106 INGOLF Dining Chair         3999
       101 KALLAX Shelf Unit           4999
       105 MICKE Desk                  5999
       104 POANG Armchair              6999
       109 APPLARO Outdoor Table       8999
       102 MALM Bed Frame             12999

10 rows selected.
SQL>
```

The most expensive product appears first and the cheapest product appears last.

Query 1.3b: Products from Highest to Lowest Price

```
SELECT Product_ID, Product_Name, Price
FROM Product
ORDER BY Price DESC;
```

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  ORDER BY Price DESC;

PRODUCT_ID PRODUCT_NAME                PRICE
---------- -------------------- ----------
       102 MALM Bed Frame              12999
       109 APPLARO Outdoor Table        8999
       104 POANG Armchair               6999
       105 MICKE Desk                   5999
       101 KALLAX Shelf Unit            4999
       106 INGOLF Dining Chair          3999
       103 LACK Coffee Table            2499
       108 MULIG Clothes Rack           1999
       107 FADO Table Lamp              1499
       110 FEJKA Artificial Plant        799

10 rows selected.
SQL>
```

1.4 DISTINCT

The DISTINCT keyword removes duplicate values from the result. ORDER BY is added only to display the values in a fixed order.

Categories 1 and 7 each contain two products, but every Category_ID is listed only once.

Query 1.4: Unique Category IDs

```
SELECT DISTINCT Category_ID
FROM Product
ORDER BY Category_ID;
```

```
SQL> SELECT DISTINCT Category_ID
  2  FROM Product
  3  ORDER BY Category_ID;

CATEGORY_ID
-----------
          1
          2
          4
          5
          6
          7
          8
         10

8 rows selected.
SQL>
```

2. SEARCH PRODUCTS

2.1 Search Products Based on Price

BETWEEN is inclusive, so products priced exactly 2000 or exactly 7000 would also be included.

Query 2.1: Products Priced Between ₹2,000 and ₹7,000

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Price BETWEEN 2000 AND 7000;
```

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price BETWEEN 2000 AND 7000;

PRODUCT_ID PRODUCT_NAME            PRICE
---------- ------------------ ----------
       101 KALLAX Shelf Unit        4999
       103 LACK Coffee Table        2499
       104 POANG Armchair           6999
       105 MICKE Desk               5999
       106 INGOLF Dining Chair      3999

5 rows selected.
SQL>
```

2.2 Search Products Based on Category

The IN operator checks whether Category_ID matches any one of the values in the list.

Query 2.2: Products in Category 1, 5 and 7

```
SELECT Product_ID, Product_Name, Category_ID
FROM Product
WHERE Category_ID IN (1, 5, 7);
```

```
SQL> SELECT Product_ID, Product_Name, Category_ID
  2  FROM Product
  3  WHERE Category_ID IN (1, 5, 7);

PRODUCT_ID PRODUCT_NAME       CATEGORY_ID
---------- ----------------- -----------
       101 KALLAX Shelf Unit           7
       103 LACK Coffee Table           1
       104 POANG Armchair              1
       105 MICKE Desk                  5
       108 MULIG Clothes Rack          7

5 rows selected.
SQL>
```

2.3 Search Products Based on Availability

Availability is calculated from Inventory.Stock_Quantity by joining Product and Inventory on Product_ID. A product is available when Stock_Quantity is greater than 0 and unavailable when it is 0.

Only products with a stock quantity above zero are listed.

Query 2.3a: Products Currently Available in Stock

```
SELECT p.Product_ID, p.Product_Name, i.Stock_Quantity
FROM Product p
JOIN Inventory i
ON p.Product_ID = i.Product_ID
WHERE i.Stock_Quantity > 0;
```

```
SQL> SELECT p.Product_ID, p.Product_Name, i.Stock_Quantity
  2  FROM Product p
  3  JOIN Inventory i
  4  ON p.Product_ID = i.Product_ID
  5  WHERE i.Stock_Quantity > 0;

PRODUCT_ID PRODUCT_NAME          STOCK_QUANTITY
---------- -------------------- --------------
       101 KALLAX Shelf Unit                25
       102 MALM Bed Frame                   18
       103 LACK Coffee Table                20
       104 POANG Armchair                   32
       105 MICKE Desk                       15
       107 FADO Table Lamp                  40
       108 MULIG Clothes Rack               12
       109 APPLARO Outdoor Table            27

8 rows selected.
SQL>
```

Only products whose stock quantity is zero are listed.

Query 2.3b: Products Currently Unavailable

```
SELECT p.Product_ID, p.Product_Name, i.Stock_Quantity
FROM Product p
JOIN Inventory i
ON p.Product_ID = i.Product_ID
WHERE i.Stock_Quantity = 0;
```

```
SQL> SELECT p.Product_ID, p.Product_Name, i.Stock_Quantity
  2  FROM Product p
  3  JOIN Inventory i
  4  ON p.Product_ID = i.Product_ID
  5  WHERE i.Stock_Quantity = 0;

PRODUCT_ID PRODUCT_NAME           STOCK_QUANTITY
---------- -------------------- --------------
       106 INGOLF Dining Chair                0
       110 FEJKA Artificial Plant             0

2 rows selected.
SQL>
```

3. RETRIEVE CUSTOMER AND PRODUCT INFORMATION

3.1 Display All Customers

This query displays every column and every row of the Customer table.

3.2 Display Product Information

This query retrieves selected information about every product stored in the Product table.

Query 3.2: Product Names and Prices

```
SELECT Product_Name, Price
FROM Product;
```

```
SQL> SELECT Product_Name, Price
  2  FROM Product;

PRODUCT_NAME                PRICE
--------------------- ----------
KALLAX Shelf Unit            4999
MALM Bed Frame              12999
LACK Coffee Table            2499
POANG Armchair               6999
MICKE Desk                   5999
INGOLF Dining Chair          3999
FADO Table Lamp              1499
MULIG Clothes Rack           1999
APPLARO Outdoor Table        8999
FEJKA Artificial Plant        799

10 rows selected.
SQL>
```

3.3 Search for a Particular Product

Product_ID uniquely identifies a product, so searching with Product_ID = 101 returns the single KALLAX Shelf Unit record.

Query 3.3: Search for Product 101

```
SELECT Product_ID, Product_Name, Price, Category_ID
FROM Product
WHERE Product_ID = 101;
```

```
SQL> SELECT Product_ID, Product_Name, Price, Category_ID
  2  FROM Product
  3  WHERE Product_ID = 101;

PRODUCT_ID PRODUCT_NAME           PRICE CATEGORY_ID
---------- ----------------- ---------- -----------
       101 KALLAX Shelf Unit       4999           7

1 row selected.
SQL>
```

4. APPLY FILTERING CONDITIONS

SQL filtering operators allow records to be selected based on one or more conditions. The following subsections demonstrate the commonly used operators on the Product table.

4.1 AND Operator

AND returns a row only when both conditions are true.

Query 4.1: Price at Least ₹2,000 and at Most ₹7,000

```
SELECT Product_Name, Price, Category_ID
FROM Product
WHERE Price >= 2000 AND Price <= 7000;
```

```
SQL> SELECT Product_Name, Price, Category_ID
  2  FROM Product
  3  WHERE Price >= 2000 AND Price <= 7000;

PRODUCT_NAME             PRICE CATEGORY_ID
------------------- ---------- -----------
KALLAX Shelf Unit         4999           7
LACK Coffee Table         2499           1
POANG Armchair            6999           1
MICKE Desk                5999           5
INGOLF Dining Chair       3999           4

5 rows selected.
SQL>
```

4.2 OR Operator

OR returns a row when at least one of the conditions is true.

Query 4.2: Category 1 or Category 7

```
SELECT Product_ID, Product_Name, Category_ID
FROM Product
WHERE Category_ID = 1 OR Category_ID = 7;
```

```
SQL> SELECT Product_ID, Product_Name, Category_ID
  2  FROM Product
  3  WHERE Category_ID = 1 OR Category_ID = 7;

PRODUCT_ID PRODUCT_NAME       CATEGORY_ID
---------- ------------------ -----------
       101 KALLAX Shelf Unit            7
       103 LACK Coffee Table            1
       104 POANG Armchair               1
       108 MULIG Clothes Rack           7

4 rows selected.
SQL>
```

4.3 NOT Operator

NOT reverses a condition, so every product outside Category_ID 1 is returned.

Query 4.3: Products Not in Category 1

```
SELECT Product_ID, Product_Name, Category_ID
FROM Product
WHERE NOT Category_ID = 1;
```

```
SQL> SELECT Product_ID, Product_Name, Category_ID
  2  FROM Product
  3  WHERE NOT Category_ID = 1;

PRODUCT_ID PRODUCT_NAME          CATEGORY_ID
---------- --------------------- -----------
       101 KALLAX Shelf Unit               7
       102 MALM Bed Frame                  2
       105 MICKE Desk                      5
       106 INGOLF Dining Chair             4
       107 FADO Table Lamp                 8
       108 MULIG Clothes Rack              7
       109 APPLARO Outdoor Table           6
       110 FEJKA Artificial Plant         10

8 rows selected.
SQL>
```

)

4.4 BETWEEN Operator

BETWEEN includes both boundary values, so prices of exactly 1000 and exactly 5000 would also be returned.

Query 4.4: Products Priced Between ₹1,000 and ₹5,000

```
SELECT Product_Name, Price
FROM Product
WHERE Price BETWEEN 1000 AND 5000;
```

```
SQL> SELECT Product_Name, Price
  2  FROM Product
  3  WHERE Price BETWEEN 1000 AND 5000;

PRODUCT_NAME            PRICE
------------------ ----------
KALLAX Shelf Unit         4999
LACK Coffee Table         2499
INGOLF Dining Chair       3999
FADO Table Lamp           1499
MULIG Clothes Rack        1999

5 rows selected.
SQL>
```

4.5 IN Operator

IN is a shorter way of writing several OR conditions on the same column.

Query 4.5: Category 1, 4, 5 and 7

```
SELECT Product_ID, Product_Name, Category_ID
FROM Product
WHERE Category_ID IN (1, 4, 5, 7);
```

```
SQL> SELECT Product_ID, Product_Name, Category_ID
  2  FROM Product
  3  WHERE Category_ID IN (1, 4, 5, 7);

PRODUCT_ID PRODUCT_NAME        CATEGORY_ID
---------- ------------------ -----------
       101 KALLAX Shelf Unit            7
       103 LACK Coffee Table            1
       104 POANG Armchair               1
       105 MICKE Desk                   5
       106 INGOLF Dining Chair          4
       108 MULIG Clothes Rack           7

6 rows selected.
SQL>
```

4.6 LIKE Operator

LIKE searches for a pattern in text. The percent symbol (%) stands for any number of characters, including none, so %Table% matches every Product_Name that contains the word Table. In Oracle, LIKE is case-sensitive.

Query 4.6: Product Names Containing "Table"

```
SELECT Product_ID, Product_Name, Price
FROM Product
WHERE Product_Name LIKE '%Table%';
```

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Product_Name LIKE '%Table%';

PRODUCT_ID PRODUCT_NAME              PRICE
---------- -------------------- ----------
       103 LACK Coffee Table          2499
       107 FADO Table Lamp            1499
       109 APPLARO Outdoor Table      8999

3 rows selected.
SQL>
```

4.7 Comparison Operators

Comparison operators compare a column with a value: greater than (>), less than (<), greater than or equal to (>=), less than or equal to (<=) and not equal to (<>).

Query 4.7a: Greater Than (>)

```
SELECT Product_Name, Price
FROM Product
WHERE Price > 6000;
```

```
SQL> SELECT Product_Name, Price
  2  FROM Product
  3  WHERE Price > 6000;

PRODUCT_NAME               PRICE
-------------------- ----------
MALM Bed Frame             12999
POANG Armchair              6999
APPLARO Outdoor Table       8999

3 rows selected.
SQL>
```

Query 4.7b: Less Than (<)

```
SELECT Product_Name, Price
FROM Product
WHERE Price < 1500;
```

```
SQL> SELECT Product_Name, Price
  2  FROM Product
  3  WHERE Price < 1500;

PRODUCT_NAME               PRICE
-------------------- ----------
FADO Table Lamp             1499
FEJKA Artificial Plant       799

2 rows selected.
SQL>
```

Query 4.7c: Greater Than or Equal To (>=)

```
SELECT Product_Name, Price
FROM Product
WHERE Price >= 8999;
```

```
SQL> SELECT Product_Name, Price
  2  FROM Product
  3  WHERE Price >= 8999;

PRODUCT_NAME               PRICE
-------------------- ----------
MALM Bed Frame              12999
APPLARO Outdoor Table        8999

2 rows selected.
SQL>
```

Query 4.7d: Less Than or Equal To (<=)

```
SELECT Product_Name, Price
FROM Product
WHERE Price <= 2499;
```

```
SQL> SELECT Product_Name, Price
  2  FROM Product
  3  WHERE Price <= 2499;

PRODUCT_NAME                PRICE
-------------------- ----------
LACK Coffee Table            2499
FADO Table Lamp              1499
MULIG Clothes Rack           1999
FEJKA Artificial Plant        799

4 rows selected.
SQL>
```

Query 4.7e: Not Equal To

```
SELECT Product_ID, Product_Name, Category_ID
FROM Product
WHERE Category_ID <> 7;
```

```
SQL> SELECT Product_ID, Product_Name, Category_ID
  2  FROM Product
  3  WHERE Category_ID <> 7;

PRODUCT_ID PRODUCT_NAME           CATEGORY_ID
---------- -------------------- -----------
       102 MALM Bed Frame                 2
       103 LACK Coffee Table              1
       104 POANG Armchair                 1
       105 MICKE Desk                     5
       106 INGOLF Dining Chair            4
       107 FADO Table Lamp                8
       109 APPLARO Outdoor Table          6
       110 FEJKA Artificial Plant        10

8 rows selected.
SQL>
```

4.8 Combined Filtering Conditions

AND, OR and parentheses can be combined in one WHERE clause. AND is evaluated before OR, so parentheses are used to group the OR conditions and make sure they are checked together before the price condition is applied.

Query 4.8: Category 1 or 7 with Price Below ₹5,000

```
SELECT Product_ID, Product_Name, Price, Category_ID
FROM Product
WHERE (Category_ID = 1 OR Category_ID = 7)
AND Price < 5000;
```

```
SQL> SELECT Product_ID, Product_Name, Price, Category_ID
  2  FROM Product
  3  WHERE (Category_ID = 1 OR Category_ID = 7)
  4  AND Price < 5000;

PRODUCT_ID PRODUCT_NAME           PRICE CATEGORY_ID
---------- ------------------ ---------- -----------
       101 KALLAX Shelf Unit        4999           7
       103 LACK Coffee Table        2499           1
       108 MULIG Clothes Rack       1999           7

3 rows selected.
SQL>
```

4.9 IS NULL and IS NOT NULL

IS NULL finds rows where a column has no value and IS NOT NULL finds rows where a value is present. Product_ID is the identifying column of the table and always contains a value, so IS NULL

returns no rows here. The same queries on a nullable column such as Description return rows only if matching data exists.

Query 4.9a: IS NULL

```
SELECT Product_ID, Product_Name
FROM Product
WHERE Product_ID IS NULL;
```

```
SQL> SELECT Product_ID, Product_Name
  2  FROM Product
  3  WHERE Product_ID IS NULL;

no rows selected
SQL>
```

Query 4.9b: IS NOT NULL

```
SELECT Product_ID, Product_Name
FROM Product
WHERE Product_ID IS NOT NULL;
```

```
SQL> SELECT Product_ID, Product_Name
  2  FROM Product
  3  WHERE Product_ID IS NOT NULL;

PRODUCT_ID PRODUCT_NAME
---------- ----------------------
       101 KALLAX Shelf Unit
       102 MALM Bed Frame
       103 LACK Coffee Table
       104 POANG Armchair
       105 MICKE Desk
       106 INGOLF Dining Chair
       107 FADO Table Lamp
       108 MULIG Clothes Rack
       109 APPLARO Outdoor Table
       110 FEJKA Artificial Plant

10 rows selected.
SQL>
```

4.10 Summary of Filtering Operators

Operator	Purpose	Example
WHERE	Filters rows by a condition	`WHERE Price < 2000`
AND	Both conditions must be true	`WHERE Price >= 2000 AND Price <= 7000`
OR	At least one condition must be true	`WHERE Category_ID = 1 OR Category_ID = 7`
NOT	Reverses a condition	`WHERE NOT Category_ID = 1`
BETWEEN	Value lies in a range (inclusive)	`WHERE Price BETWEEN 1000 AND 5000`
IN	Value matches any item in a list	`WHERE Category_ID IN (1, 4, 5, 7)`
LIKE	Matches a text pattern	`WHERE Product_Name LIKE '%Table%'`
>, <, >=, <=, <>	Compares a column with a value	`WHERE Category_ID <> 7`

Operator	Purpose	Example
IS NULL / IS NOT NULL	Tests for a missing / present value	`WHERE Product_ID IS NOT NULL`

5. BASIC BUSINESS REPORTS

5.1 Product Price Report

Purpose: to list the products from the most expensive to the cheapest.

The report helps management identify premium and low-cost products quickly.

Query 5.1: Product Price Report

```
SELECT Product_ID, Product_Name, Category_ID, Price
FROM Product
ORDER BY Price DESC;
```

```
SQL> SELECT Product_ID, Product_Name, Category_ID, Price
  2  FROM Product
  3  ORDER BY Price DESC;

PRODUCT_ID PRODUCT_NAME           CATEGORY_ID      PRICE
---------- ---------------------- ----------- ----------
       102 MALM Bed Frame                   2      12999
       109 APPLARO Outdoor Table            6       8999
       104 POANG Armchair                   1       6999
       105 MICKE Desk                       5       5999
       101 KALLAX Shelf Unit                7       4999
       106 INGOLF Dining Chair              4       3999
       103 LACK Coffee Table                1       2499
       108 MULIG Clothes Rack               7       1999
       107 FADO Table Lamp                  8       1499
       110 FEJKA Artificial Plant          10        799

10 rows selected.
SQL>
```

5.2 Product Count by Category

Purpose: to show how many products belong to each category.

COUNT(*) counts the rows in each Category_ID group.

Query 5.2: Product Count by Category

```
SELECT Category_ID, COUNT(*) AS Product_Count
FROM Product
GROUP BY Category_ID
ORDER BY Category_ID;
```

```
SQL> SELECT Category_ID, COUNT(*) AS Product_Count
  2  FROM Product
  3  GROUP BY Category_ID
  4  ORDER BY Category_ID;

CATEGORY_ID PRODUCT_COUNT
----------- -------------
          1             2
          2             1
          4             1
          5             1
          6             1
          7             2
          8             1
         10             1

8 rows selected.
SQL>
```

5.3 Average Product Price by Category

Purpose: to compare the average price level of each category.

AVG calculates the mean price of each group and ROUND limits it to two decimal places.

Query 5.3: Average Price by Category

```
SELECT Category_ID, ROUND(AVG(Price), 2) AS Avg_Price
FROM Product
GROUP BY Category_ID
ORDER BY Category_ID;
```

```
SQL> SELECT Category_ID, ROUND(AVG(Price), 2) AS Avg_Price
  2  FROM Product
  3  GROUP BY Category_ID
  4  ORDER BY Category_ID;

CATEGORY_ID  AVG_PRICE
----------- ----------
          1       4749
          2      12999
          4       3999
          5       5999
          6       8999
          7       3499
          8       1499
         10        799

8 rows selected.
SQL>
```

5.4 Inventory and Availability Report

Purpose: to show the stock level of each product and summarise how many products are available or unavailable.

Query 5.4a: Product Stock Quantities

```
SELECT p.Product_Name, i.Stock_Quantity,
CASE WHEN i.Stock_Quantity > 0 THEN 'Available'
ELSE 'Unavailable' END AS Availability
FROM Product p
JOIN Inventory i
ON p.Product_ID = i.Product_ID;
```

```
SQL> SELECT p.Product_Name, i.Stock_Quantity,
  2  CASE WHEN i.Stock_Quantity > 0 THEN 'Available'
  3  ELSE 'Unavailable' END AS Availability
  4  FROM Product p
  5  JOIN Inventory i
  6  ON p.Product_ID = i.Product_ID;

PRODUCT_NAME          STOCK_QUANTITY AVAILABILITY
--------------------- -------------- ------------
KALLAX Shelf Unit                 25 Available
MALM Bed Frame                    18 Available
LACK Coffee Table                 20 Available
POANG Armchair                    32 Available
MICKE Desk                        15 Available
INGOLF Dining Chair                0 Unavailable
FADO Table Lamp                   40 Available
MULIG Clothes Rack                12 Available
APPLARO Outdoor Table             27 Available
FEJKA Artificial Plant             0 Unavailable

10 rows selected.
SQL>
```

Query 5.4b: Availability Summary

```
SELECT
SUM(CASE WHEN Stock_Quantity > 0
THEN 1 ELSE 0 END) AS Available_Products,
SUM(CASE WHEN Stock_Quantity = 0
THEN 1 ELSE 0 END) AS Unavailable_Products
FROM Inventory;
```

```
SQL> SELECT
  2  SUM(CASE WHEN Stock_Quantity > 0
  3  THEN 1 ELSE 0 END) AS Available_Products,
  4  SUM(CASE WHEN Stock_Quantity = 0
  5  THEN 1 ELSE 0 END) AS Unavailable_Products
  6  FROM Inventory;

AVAILABLE_PRODUCTS UNAVAILABLE_PRODUCTS
------------------ --------------------
                 8                    2

1 row selected.
SQL>
```

5.5 Total Stock Report

Purpose: to find the total number of units held in inventory.

SUM adds the Stock_Quantity values of all inventory records.

Query 5.5: Total Stock Quantity

```
SELECT SUM(Stock_Quantity) AS Total_Stock
FROM Inventory;
```

```
SQL> SELECT SUM(Stock_Quantity) AS Total_Stock
  2  FROM Inventory;

TOTAL_STOCK
-----------
        189

1 row selected.
SQL>
```

5.6 Five Highest-Priced Products

Purpose: to list the five most expensive products. The rows are sorted by price in descending order and FETCH FIRST limits the output to five rows (supported in Oracle 12c and later, including 21c XE).

Query 5.6: Five Highest-Priced Products

```
SELECT Product_ID, Product_Name, Price
FROM Product
ORDER BY Price DESC
FETCH FIRST 5 ROWS ONLY;
```

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  ORDER BY Price DESC
  4  FETCH FIRST 5 ROWS ONLY;

PRODUCT_ID PRODUCT_NAME               PRICE
---------- -------------------- ----------
       102 MALM Bed Frame            12999
       109 APPLARO Outdoor Table      8999
       104 POANG Armchair             6999
       105 MICKE Desk                 5999
       101 KALLAX Shelf Unit          4999

5 rows selected.
SQL>
```

6. CONCLUSION

This weekly skill task demonstrated basic SQL retrieval, sorting, filtering, product searching, inventory availability checks and business reporting on the IKEA E-Commerce Database Management System. The SELECT, WHERE, ORDER BY and DISTINCT clauses were applied to the Product table, products were searched by price, category and stock availability, and a range of filtering operators was practised. Simple reports using COUNT, AVG, SUM and GROUP BY were also produced. These queries form a foundation for the more advanced SQL work in the coming weeks